Ares Base Command

Distributed Monitoring & Automation Platform

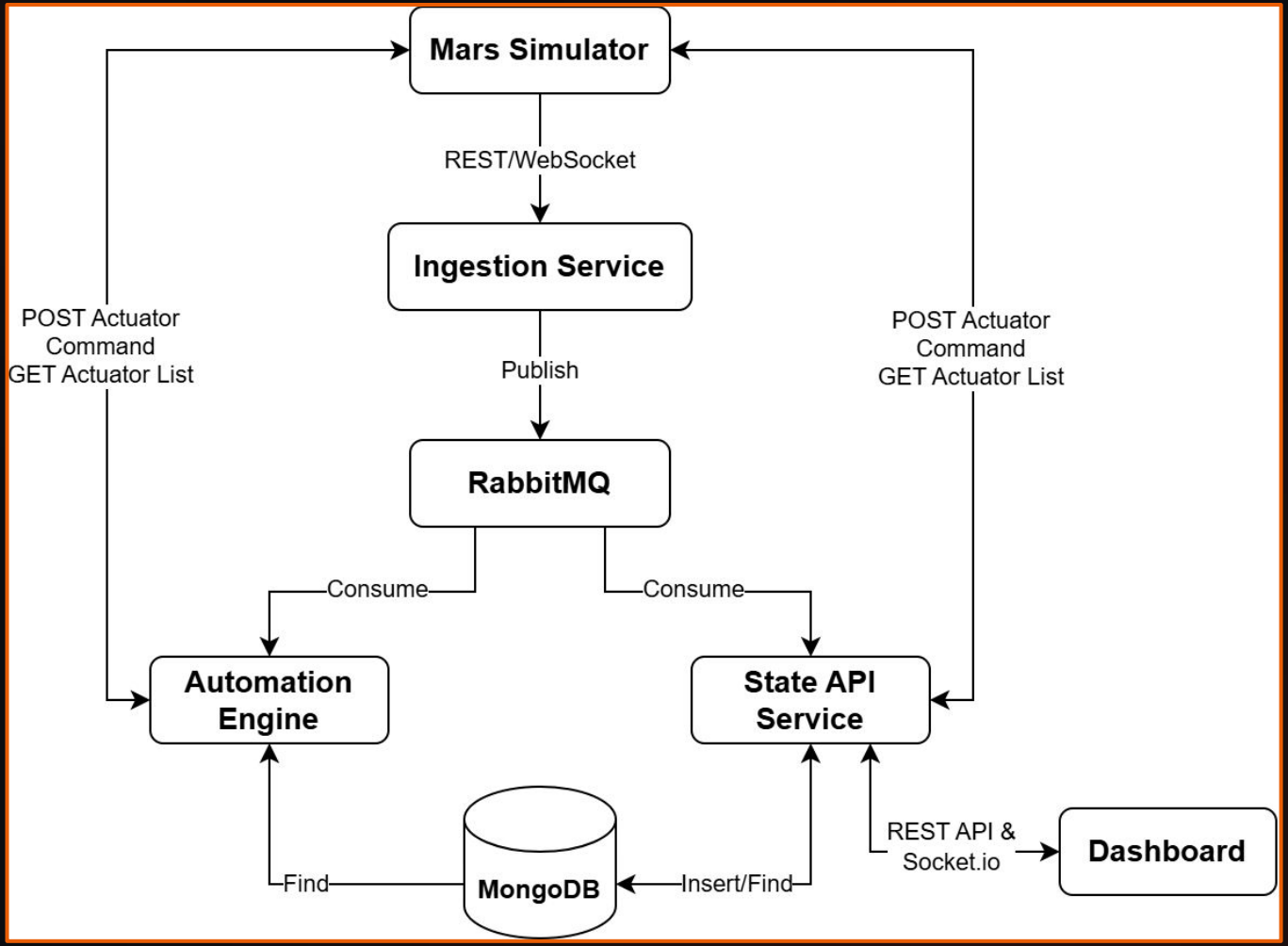
Network Architecture Overview

Infrastructure as Code (IaC)

Docker Compose: to orchestrate independent containers. Internal Docker Network useful to establish inter-container connections.

Event-Driven Core: RabbitMQ Pub/Sub (Fanout) completely decouples microservices, eliminating synchronous REST bottlenecks.

Ports open to the host:
- 8080: Mars Simulator
- 3001: State API (UI/WebSockets)
- 5672 / 27017: RabbitMQ & MongoDB



Docker Containers

Ingestion Service

Node.js/TypeScript service handling REST polling and WebSockets. Normalizes data and publishes to RabbitMQ fanout exchanges.

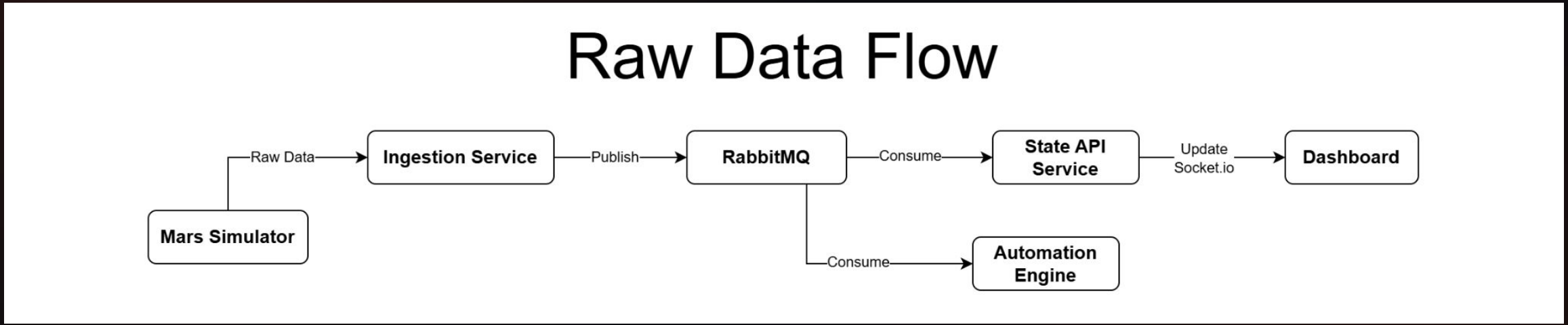
Automation Engine

Node.js/TypeScript service evaluating events against IF-THEN rules. Rules are persisted in MongoDB to survive restarts.

State API Service

State API serves as BFF. React SPA dashboard provides real-time WebSockets, dynamic widgets, and actuator controls.

Raw Data Flow



Data Ingestion & Normalization

- The ingestion service takes raw heterogeneous data from the Mars SImulator and outputs standardized events to the Message Broker.
- The Mars Simulator utilizes two distinct communication models: REST API Polling for low-frequency scalar sensors (Temperature, Pressure, etc...) and WebSocket Streams for high-speed, complex telemetry (Radiation, Power Bus.etc...).
- The ingestion services acts in order to isolates Ares from the chaotic and unpredictable data formats of external hardware, ensuring that external changes never "infect" or break the internal logic.

Heterogeneous Sensor Data

Sensors output varying structures requiring immediate sanitization.

```
{ "id": "air_sense_01", "type": "CO2_Level", "val": 450.5, "u": "ppm", "ts": 1710115200 }
```

Ares Unified Schema

Normalized internal standard format sent to RabbitMQ.

```
{ "device_id": "air_sense_01", "timestamp": "2024-03-11T12:00:00Z", "status": "active", "readings": [ { "metric": "co2", "value": 450.5, "unit": "ppm" } ] }
```

Automation Engine Logic

Reactive Evaluation

Dynamically and continuously evaluates incoming normalized sensor events, comparing it with active IF-THEN rules.

State Awareness

Implements an anti-spam cache. Checks the current state of an actuator before executing, ensuring we don't spam the simulator.

MongoDB Persistence

Connects directly to the Mars-DB to persist automation rules, guaranteeing rules survival over crashes or system reboots.

Execution & Alerting

Automatically dispatches REST POST requests to external physical actuators and broadcasts System Alerts back to the broker.

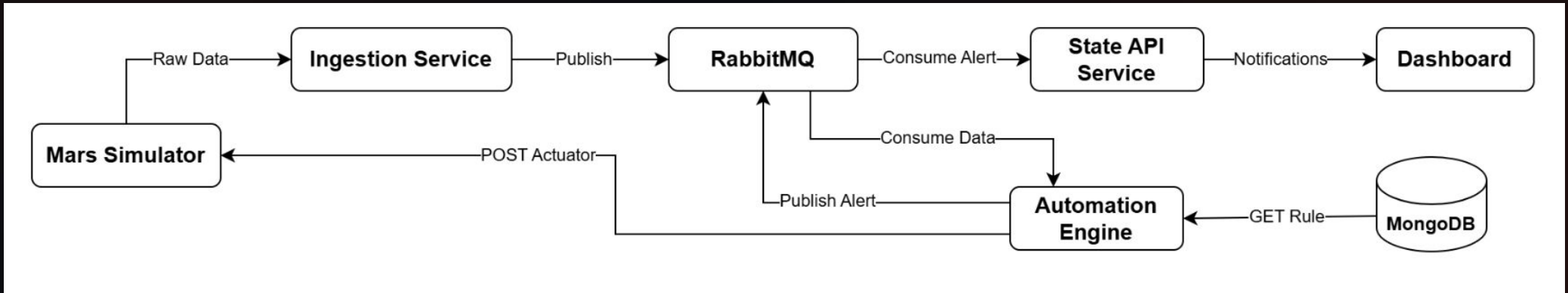
Automation Data Flow

Rule Evaluation & Filtering

1. Consume standard event from RabbitMQ.
2. Match payload against active MongoDB rules.
3. Compare sensor value vs defined threshold.
4. Check internal State Cache to prevent redundant actuation spam.

Autonomous Execution

5. If condition met & state differs, trigger action.
6. Dispatch asynchronous POST request to Simulator to toggle actuator.
7. Publish Alert to RabbitMQ, which is consumed by the State API to send Notifications to the Dashboard



State API Backend Service

Backend-For-Frontend

Acts as the BFF, bridging asynchronous RabbitMQ messages to the synchronous real-time needs of the React dashboard.

In-Memory Caching

Maintains a live dictionary of the latest sensor readings, drastically reducing network calls upon initial UI load.

WebSocket Engine

Utilizes Socket.io for full-duplex communication, pushing real-time telemetry updates and alerts instantly to the client.

React Frontend Dashboard

Reactive SPA

Built with React + Tailwind CSS for a futuristic UI. Socket.io pushes all updates without HTTP polling.

Filtered Live Streams

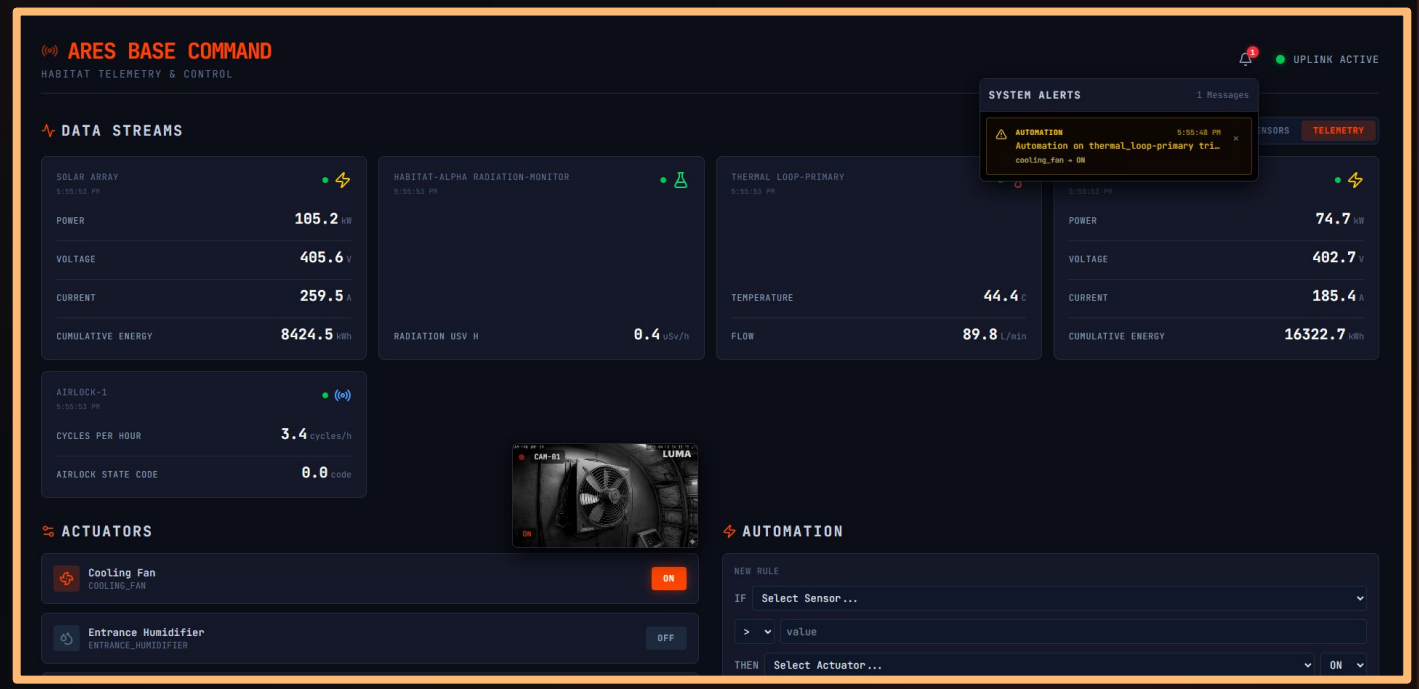
Three dedicated views: /all, /sensors (REST), /telemetry (SSE/WebSocket) for focused monitoring.

Automation & Alerts

IF-THEN are created directly on the Dashboard. Violations trigger actuator overrides and push real-time alerts.

Non-Blocking Architecture

Camera feeds and heavy media are isolated components — they never stall sensor telemetry updates.



Dynamic UI Elements

Actuator Controls

› Toggle switches send async REST POST requests to override any actuator.

› Optimistic UI: the button reflects new state instantly before server confirms.

Notification Center

› A dedicated dropdown panel displays a real-time history of system alerts triggered by the Automation Engine.

›Badge counter auto-increments; panel dismisses on outside click.

Rule Management

› An intuitive graphical form interface allows operators to dynamically Add, View, or Delete IF-THEN automation rules.

› All active rules are listed inline with an ✕ delete button.